

Klastrowanie graficznie

Stanisław Frelik

20 stycznia 2024

1 Cel pracy

Celem tej pracy jest zapoznanie się z metodami uczenia nienadzorowanego. W szczególności przeanalizujemy trzy algorytmy klastrujące, to znaczy: **KMeans**, **DBSCAN** oraz **Agglomerative Clustering**. Odpowiemy na pytanie jak parametry konkretnych metod wpływają na ich zdolność do 'znajdowania' klastrow oraz jak szum w danych 'utrudnia nam pracę'. Rozpatrzymy cztery różne sztucznie wygenerowane zbiory danych. Zapraszamy!

2 KMEANS

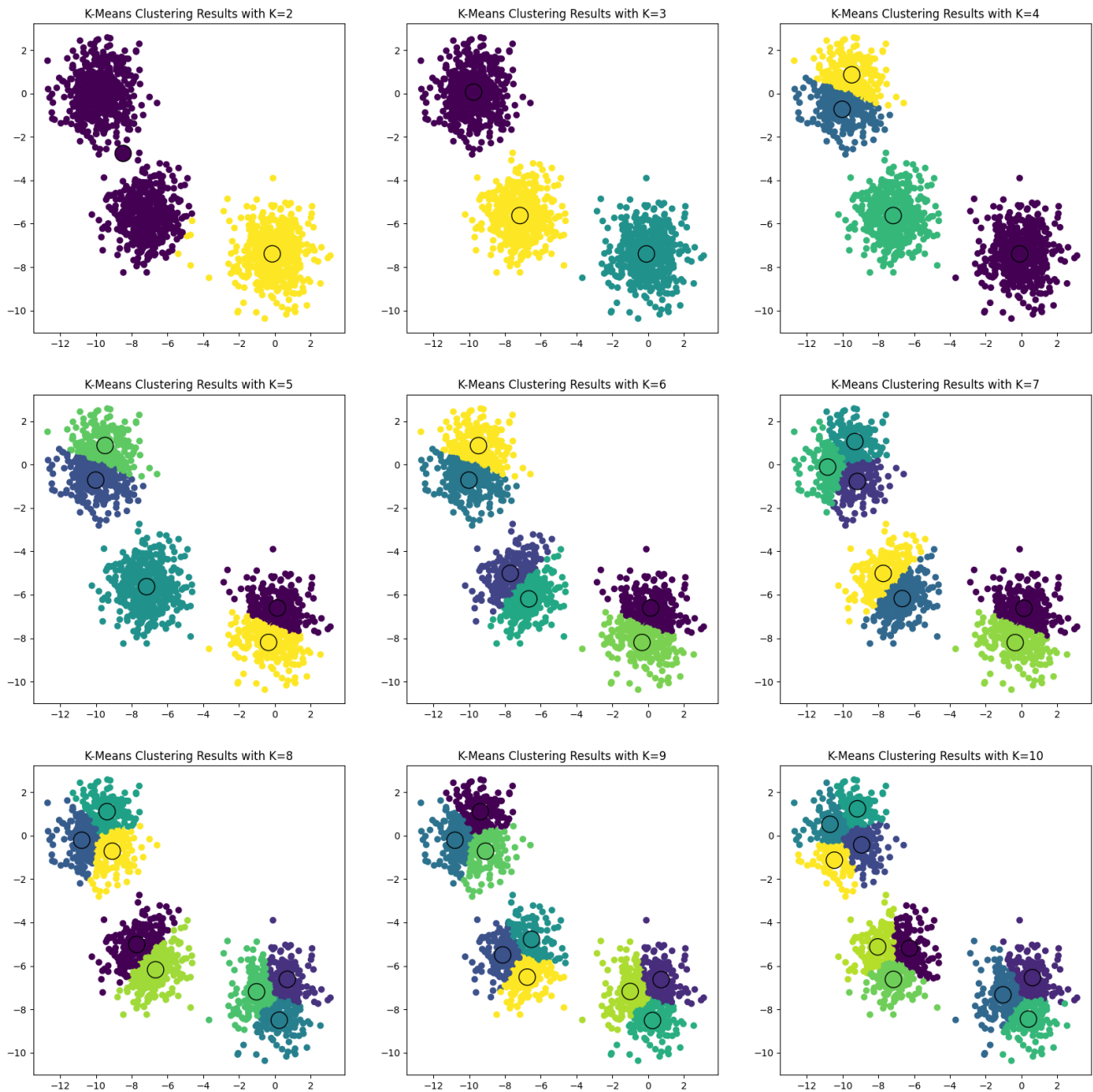
Najprostszy a zarazem wyjątkowo skuteczny algorytm klastrowania. Pomimo tego, że opiera się na losowym wyborze początkowych 'centroidów' oraz wymaga od użytkownika podania liczby klastrow, na które chcemy podzielić dane, stanowi jeden z lepszych algorytmów do 'puszczenia w ciemno'.

Na rysunkach 1-4 prezentujemy jak algorytm *KMeans* dzieli różne zbiory danych na różną liczbę klastrow (od dwóch do dziesięciu). Na wykresach zaznaczyliśmy środki ciężkości każdego z klastrow. Zauważamy, że otrzymujemy zadowalający wynik jedynie w przypadku danych typu *blobs*.

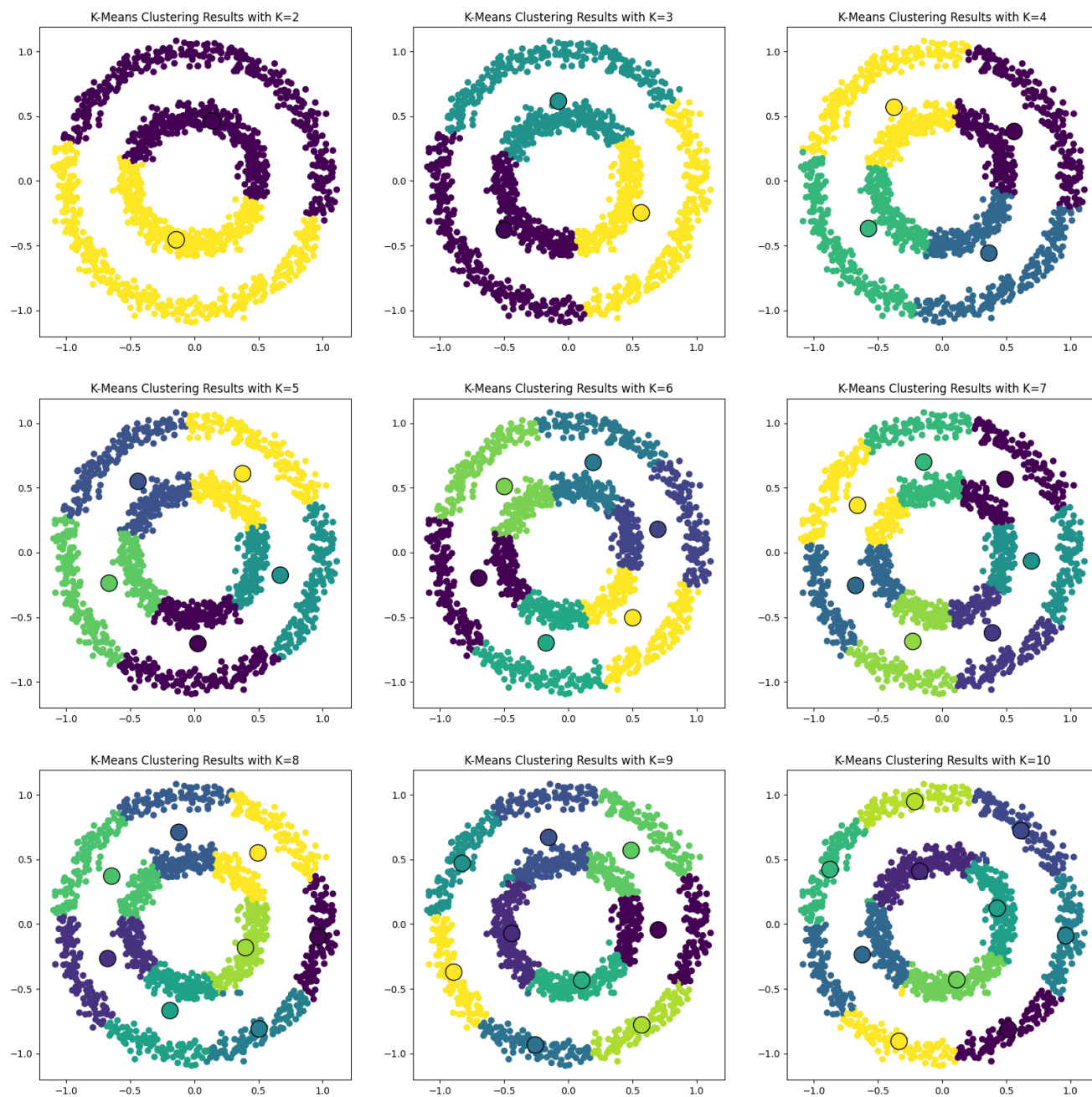
Następnie rozważymy jak parametr *noise* używany podczas generowania danych typu *circles* i *moons* wpływa na skuteczność metody *KMeans* dla różnych liczb szukanych klastrow. Oczywiście wpływu nie obserwujemy - algorytm nie radzi sobie z takimi strukturami niezależnie od tego jak bardzo są 'zaszumione'. Wizualizujemy to na rysunkach 5 i 6.

Na koniec prezentujemy wykres zależności odległości poszczególnych punktów z klastra od ich środków ciężkości względem różnych liczby klastrow dla różnych typów danych. W tym celu ustandaryzowaliśmy wszystkie zbiory danych. Na rysunku 7 widzimy wyraźnie, że algorytm *KMeans* radzi sobie wyłącznie z danymi typu *blobs*.

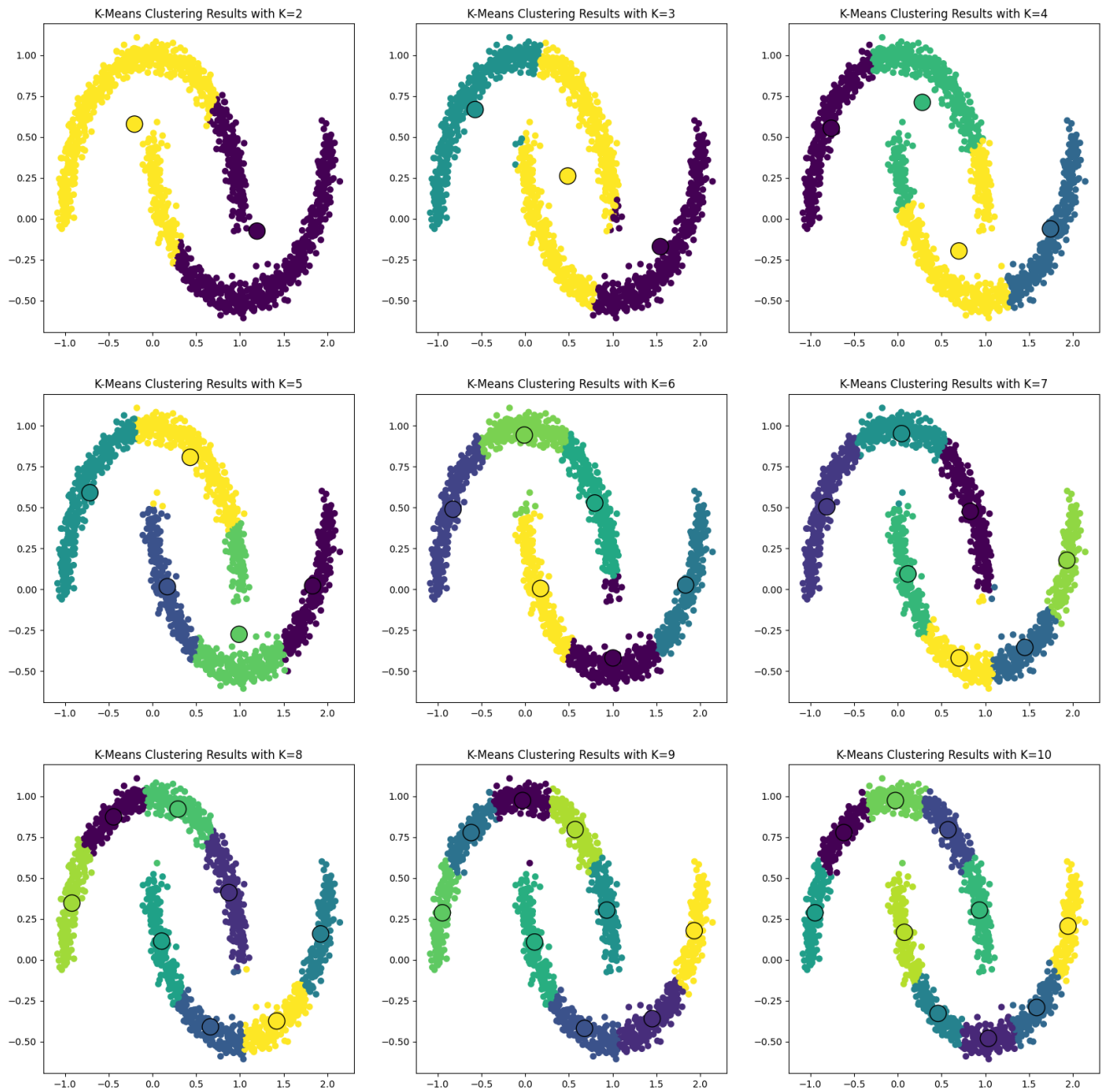
Rysunek 1: KMeans Clustering - blobs



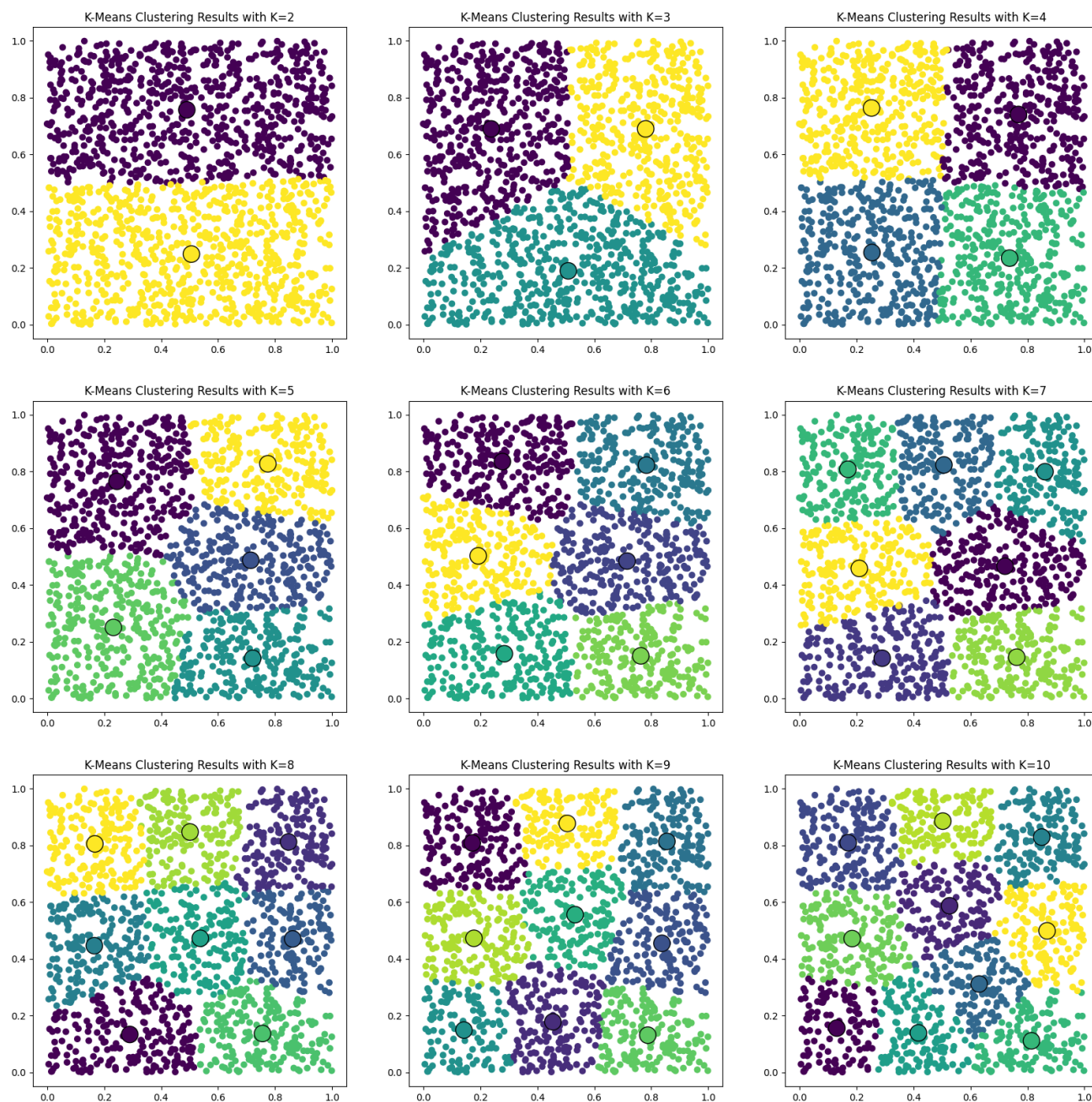
Rysunek 2: KMeans Clustering - circles



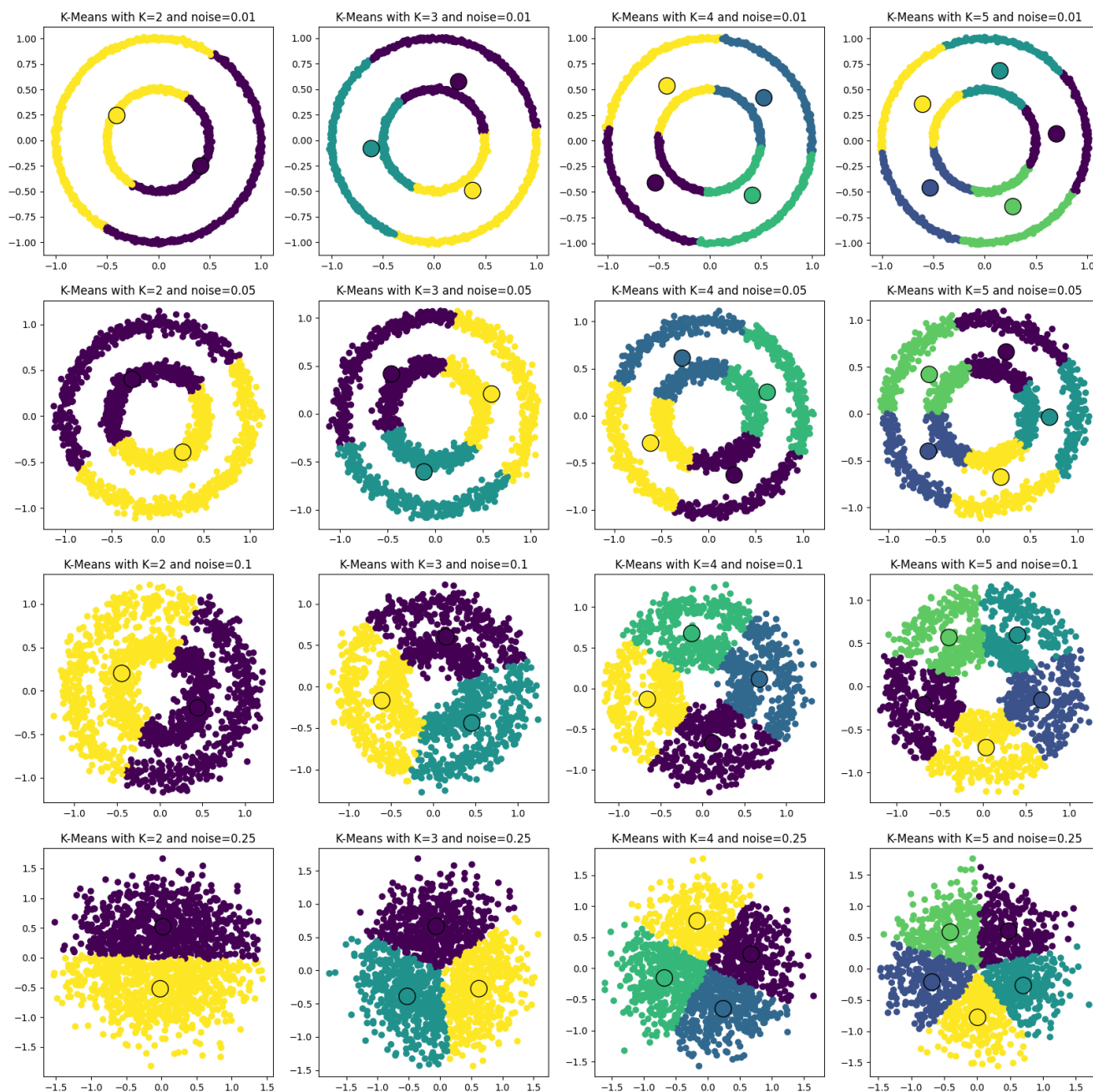
Rysunek 3: KMeans Clustering - moons



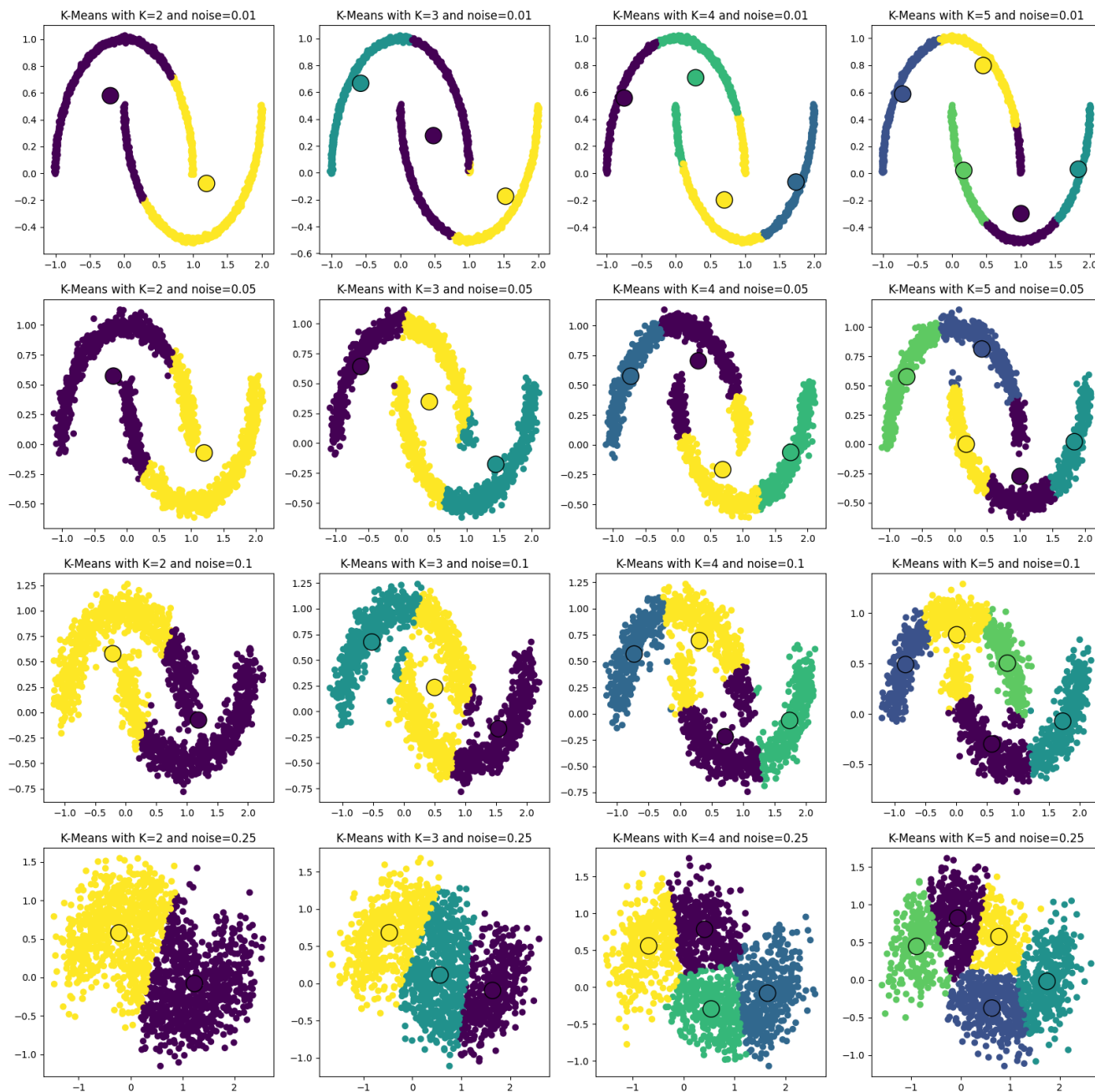
Rysunek 4: KMeans Clustering - random



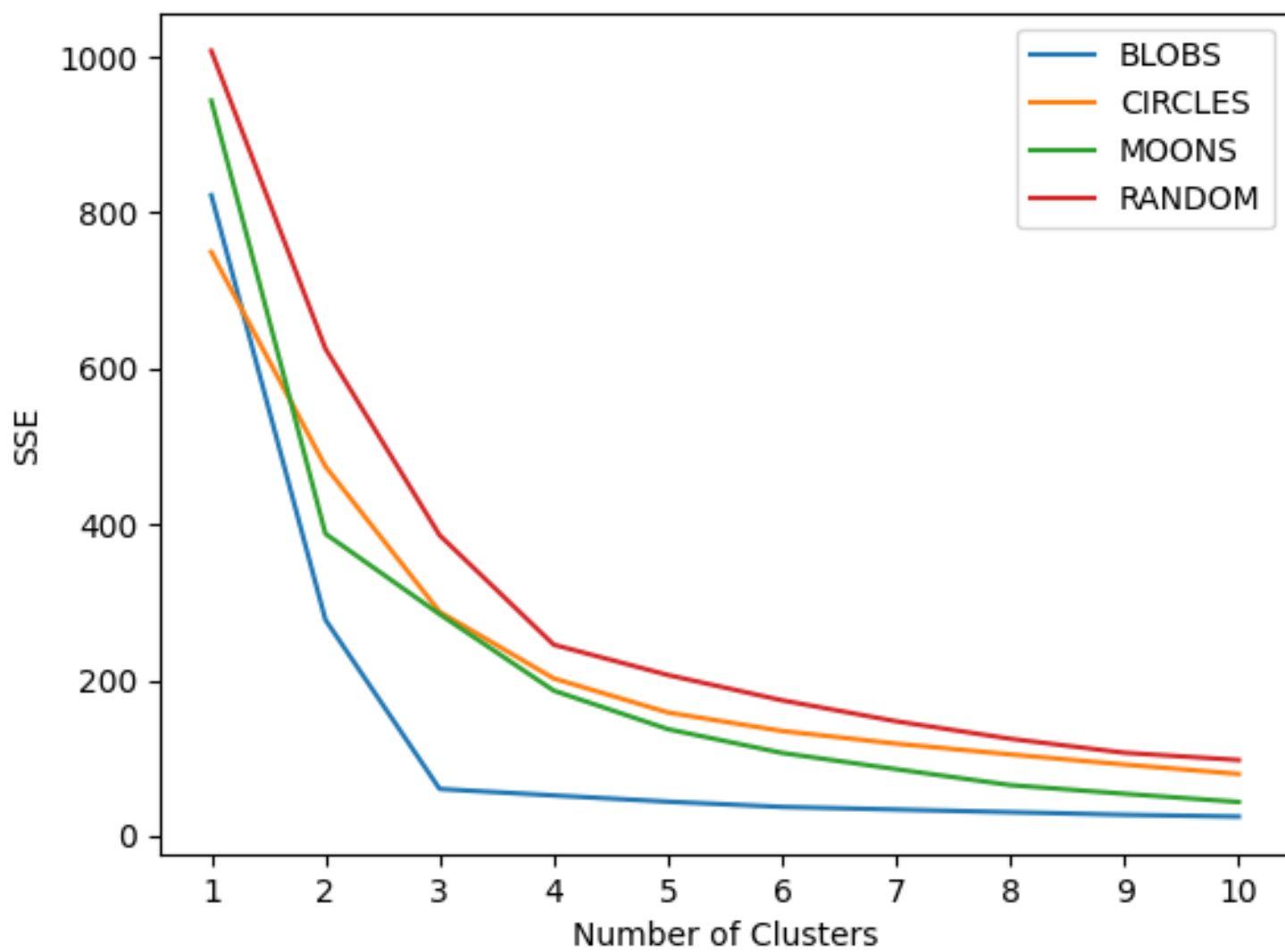
Rysunek 5: KMeans Clustering - circles with different noise



Rysunek 6: KMeans Clustering - moons with different noise



Rysunek 7: KMeans Clustering - scores



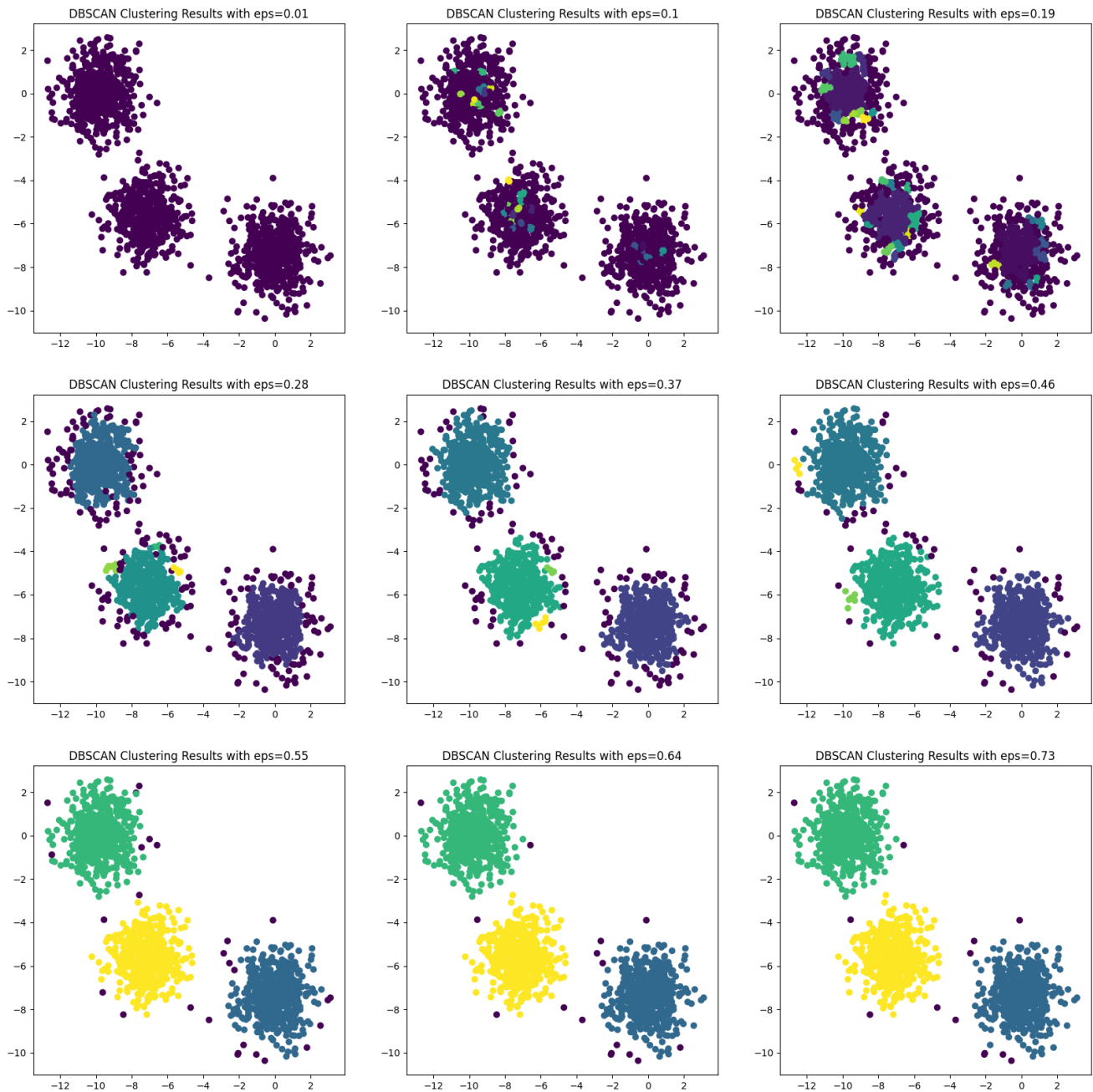
3 DBSCAN

Następnym rozważanym algorytmem jest *DBSCAN*, czyli algorytm oparty na 'gęstości'. Dokładniej polega on na wyszukiwaniu obszarów o większej gęstości poprzez łączenie w klastry odpowiednio bliskich punktów. Stąd ta metoda nie wymaga od użytkownika podania z góry szukanej liczby klastrów, jednak jej odpowiednikiem jest parametr *eps*, który sankcjonuje bliskość punktów, które finalnie trafiają do jednego klastra. Wpływ wartości tego parametru na zdolność poprawnego podziału poszczególnych zbiorów danych prezentujemy na rysunkach 8-11. Widzimy, że *DBSCAN* radzi sobie zarówno z *blobsami* jak i danymi typu *circles* i *moons*. Niemniej jest tak tylko dla specyficznych (dla tych zbiorów danych) wartości parametru *eps* (których a priori nie znamy).

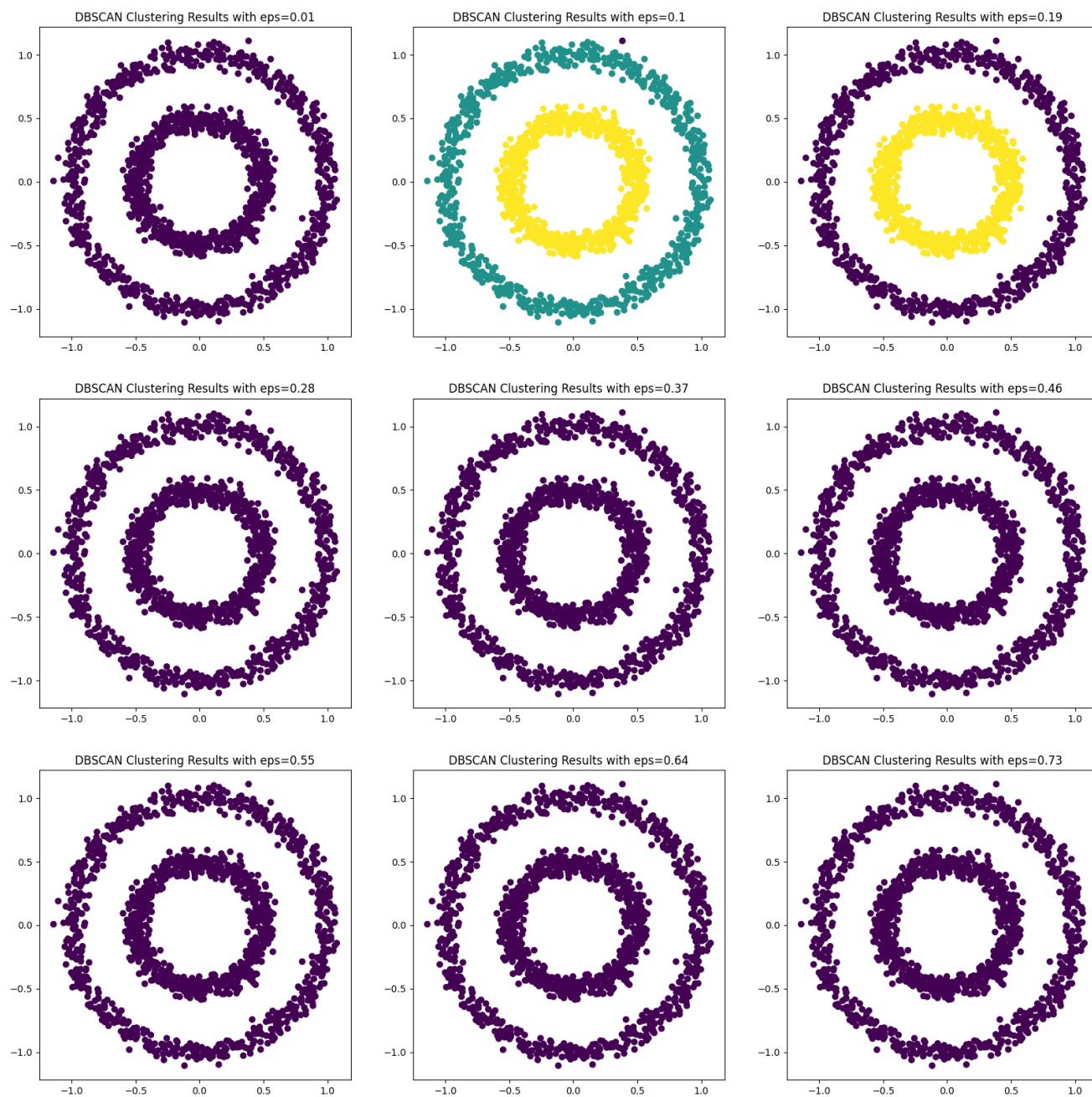
Wpływ parametru *noise* na jakość klasteryzacji przy pomocy metody *DBSCAN* jest widoczny na rysunkach 12 i 13. Widzimy, że im większy szum, tym gorsza klasteryzacja, co było do przewidzenia. Istotnie czasami obserwujemy wręcz absurdalne (lecz w pewnym sensie wizualnie atrakcyjne) klasteryzacje.

Metody *DBSCAN* i *Agglomerative Clustering* ocenimy za pomocą metryki *silhouette_score*, która wylicza średnią odległość pomiędzy punktami w każdym z klastrów. Wyniki bliskie wartości 1 oznaczają 'dobrą' klasteryzację. Im bliżej -1 tym gorzej. Natomiast wartości bliskie 0 indykują 'nakładające się' klastry. Na rysunku 14 prezentujemy otrzymane wyniki.

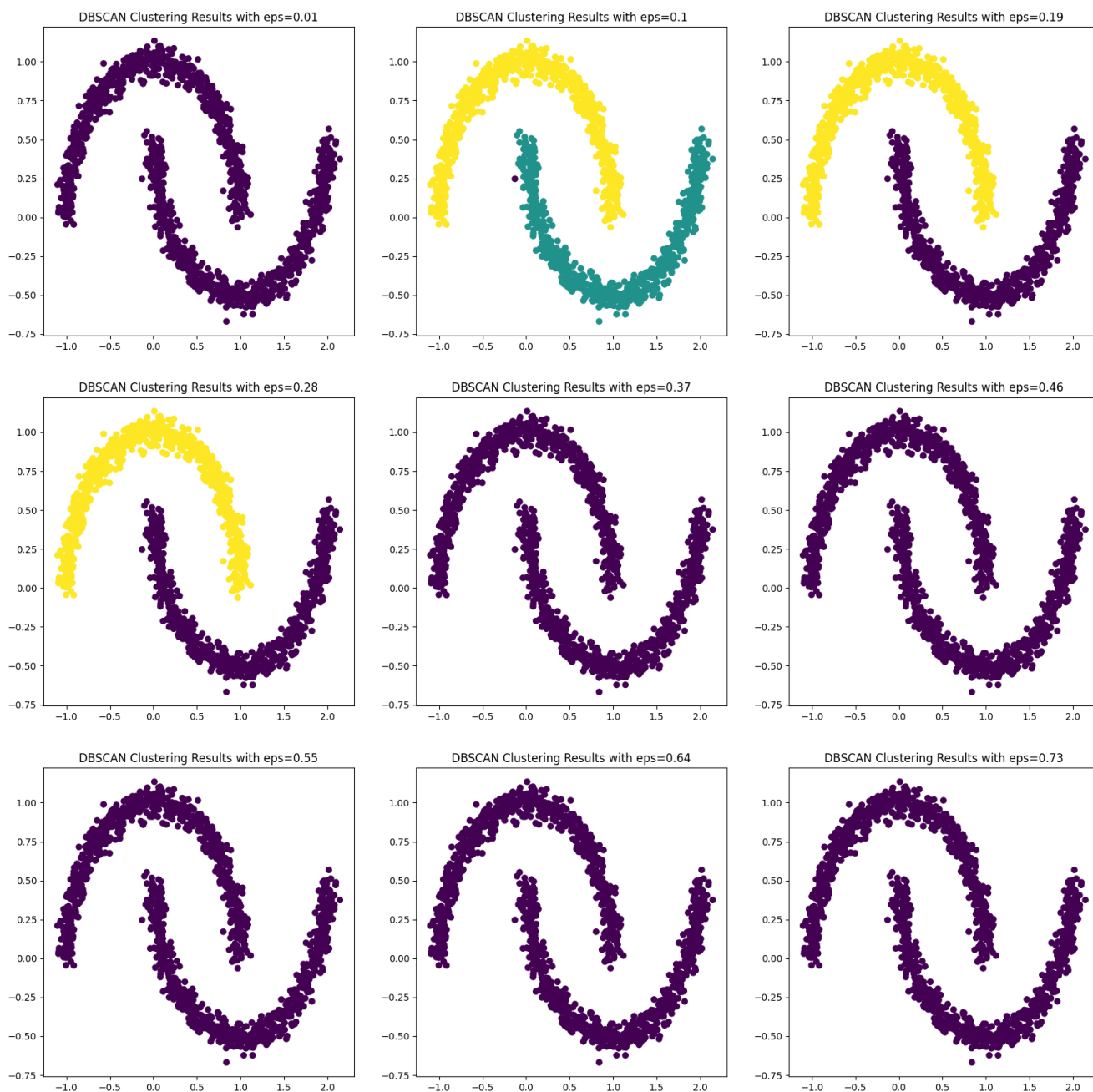
Rysunek 8: DBSCAN Clustering - blobs



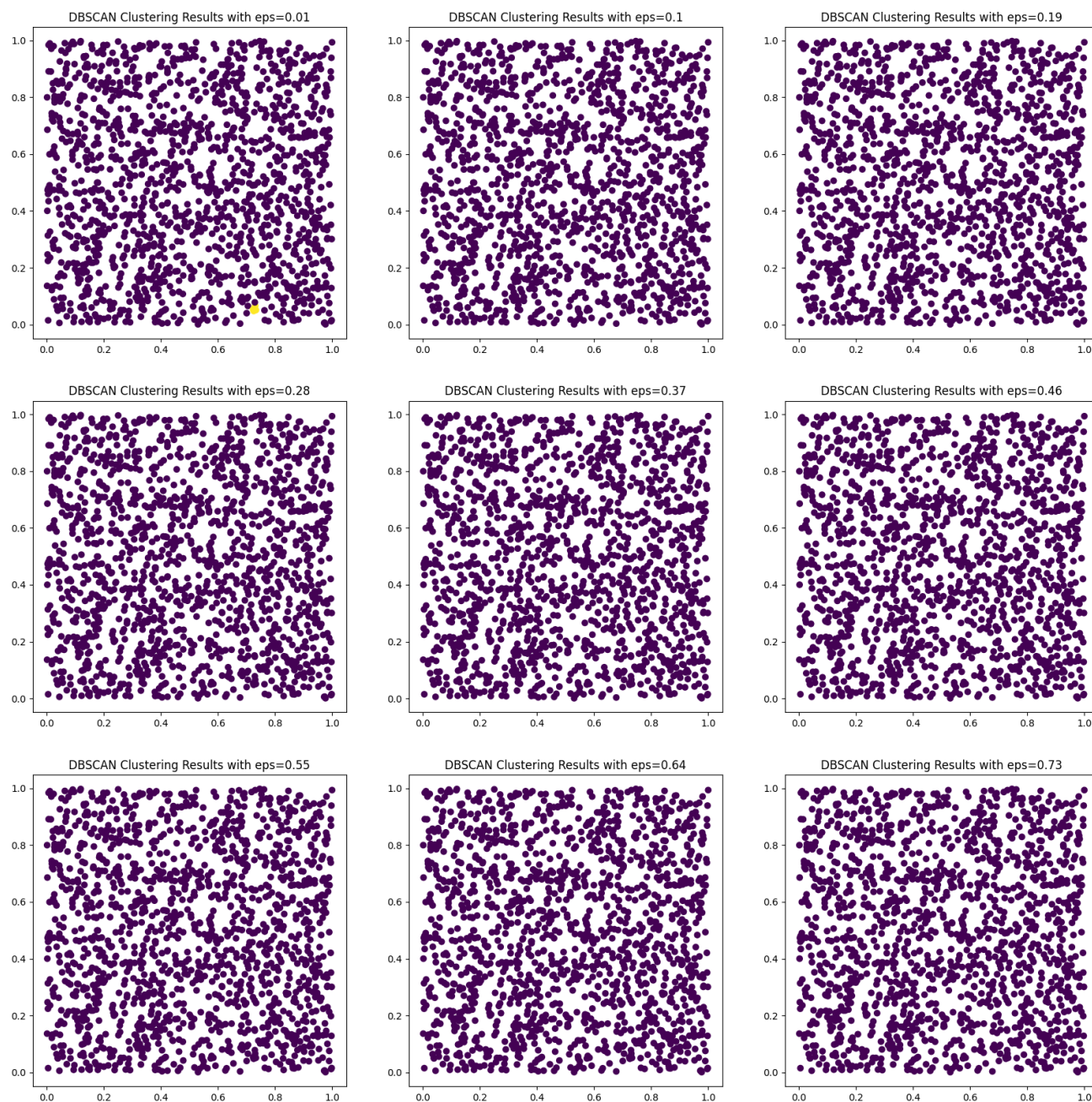
Rysunek 9: DBSCAN Clustering - circles



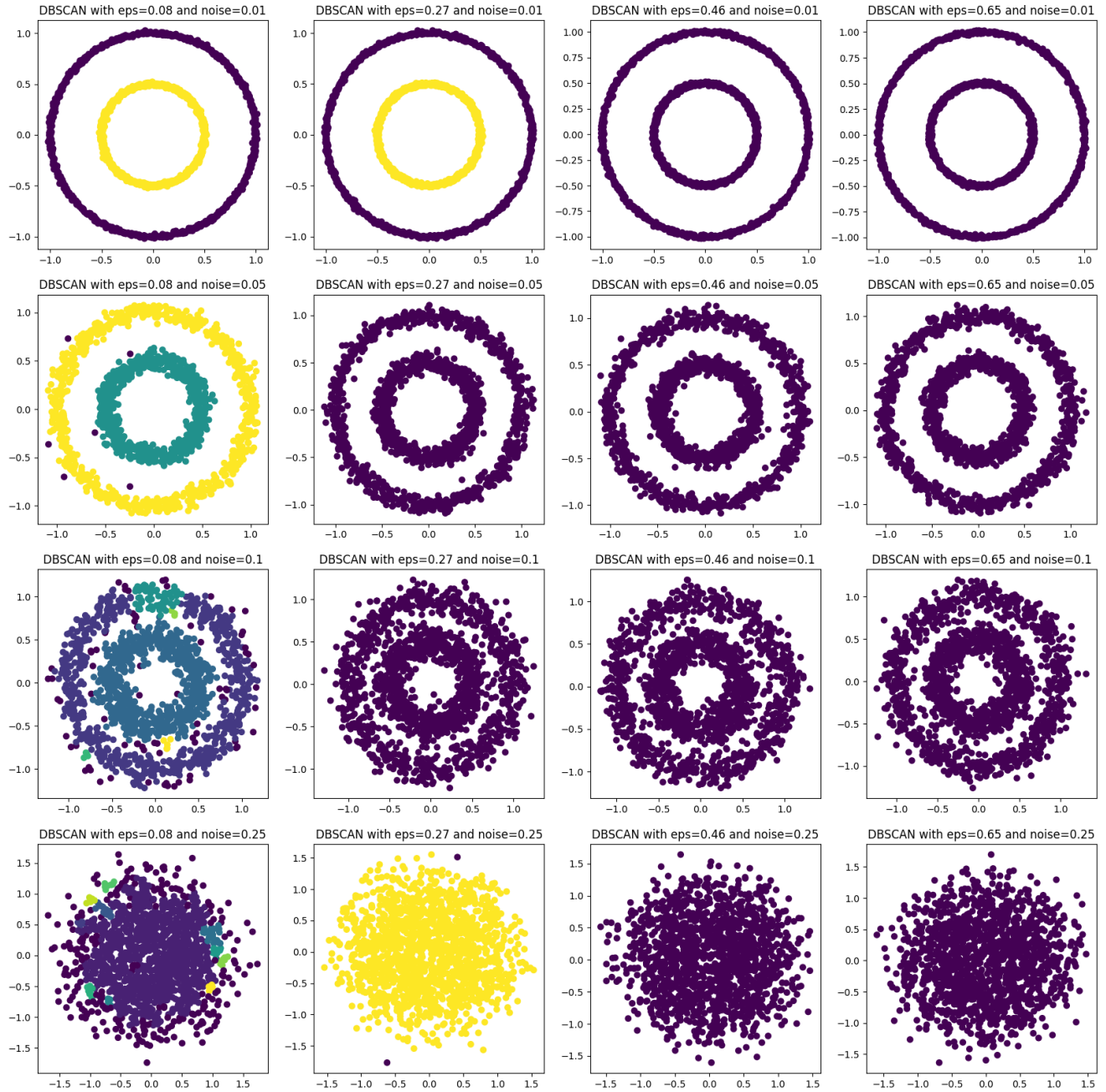
Rysunek 10: DBSCAN Clustering - moons



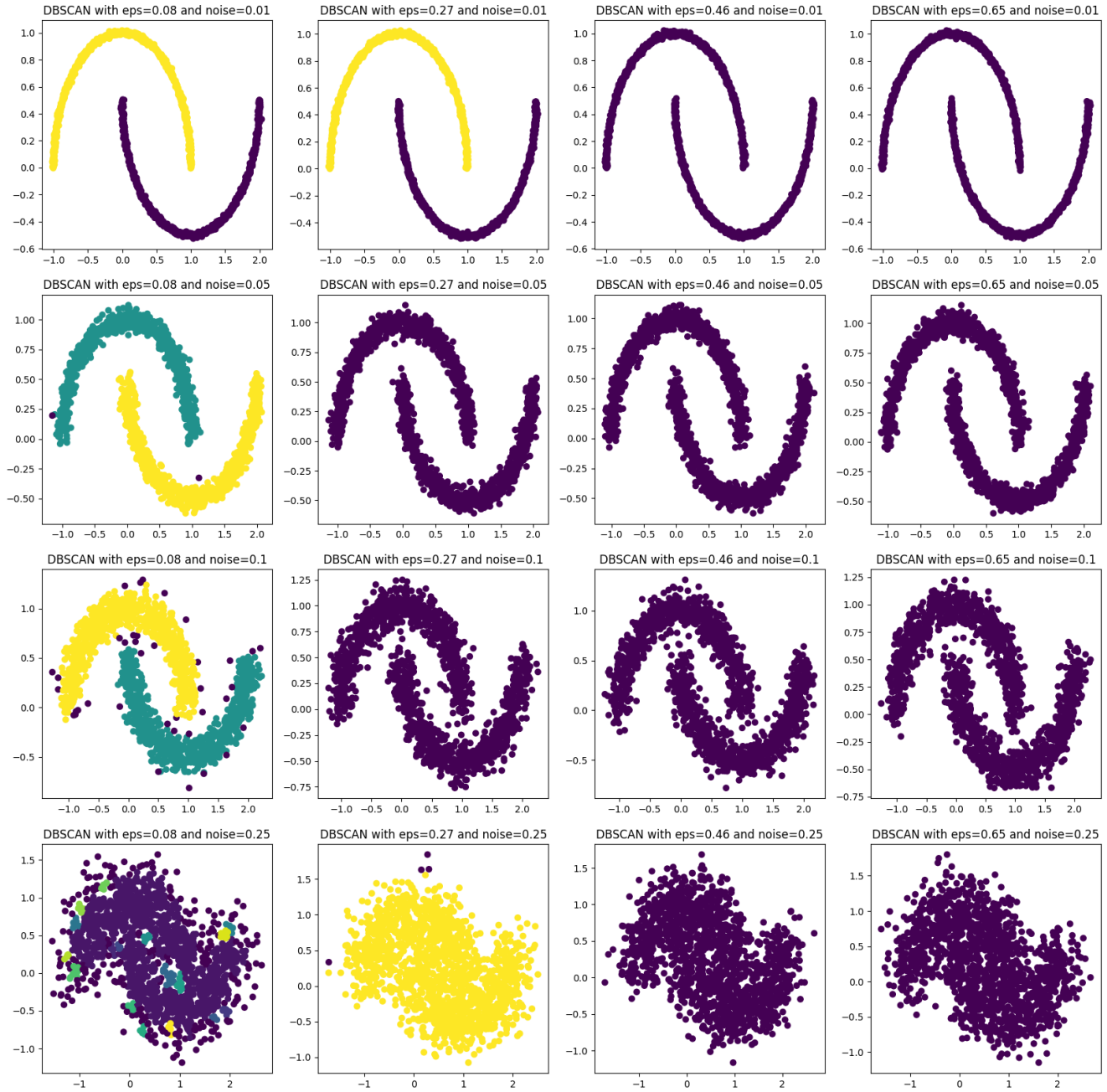
Rysunek 11: DBSCAN Clustering - random



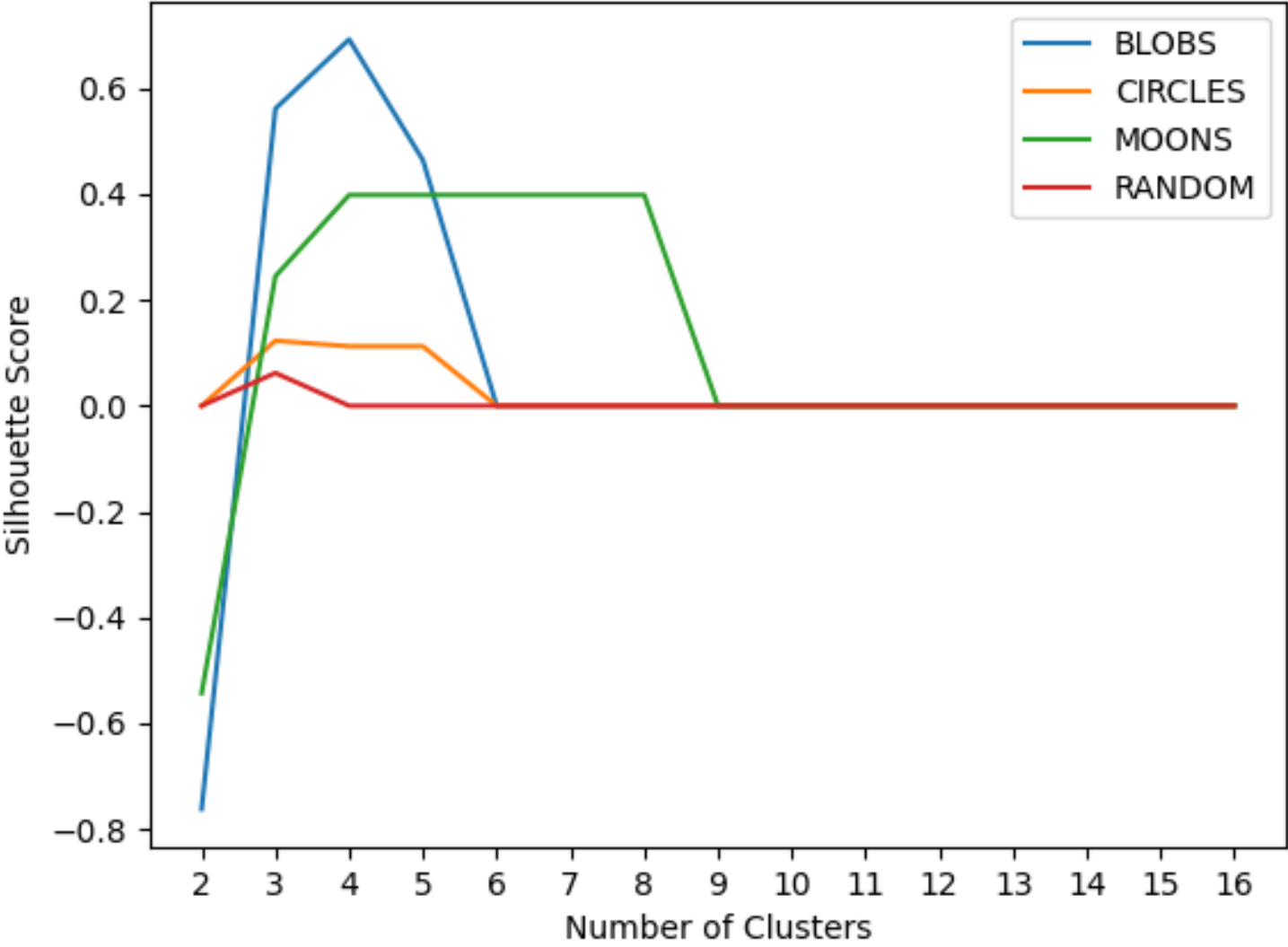
Rysunek 12: DBSCAN Clustering - circles with different noise



Rysunek 13: DBSCAN Clustering - moons with different noise



Rysunek 14: DBSCAN Clustering - score



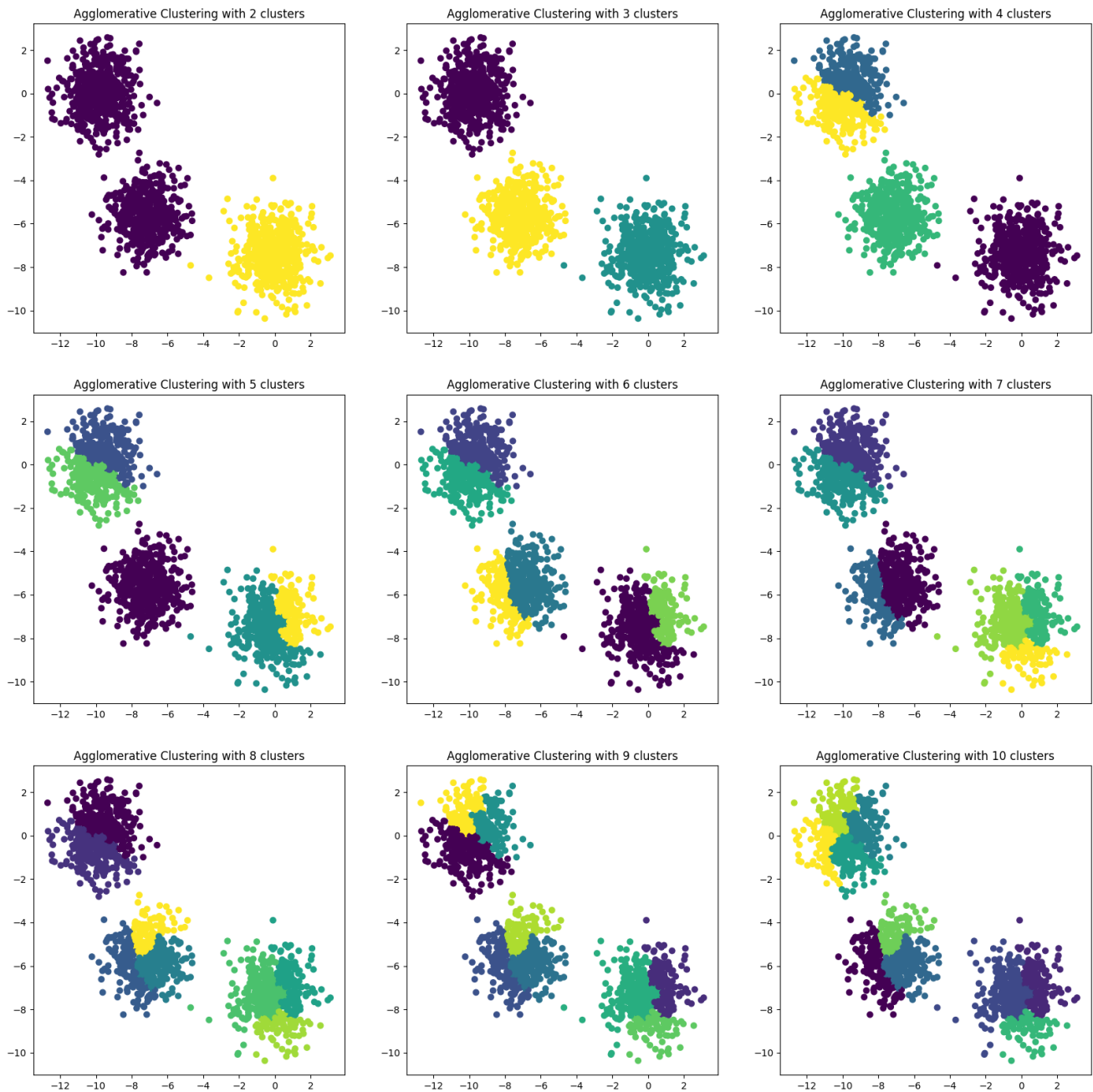
4 AGGLOMERATIVE CLUSTERING

Na koniec przeanalizujemy działanie algorytmu *Agglomerative Clustering* (znanego również jako *Hierarchical Clustering*). Metoda ta, podobnie jak *DBSCAN*, polega na łączeniu podobnych klastrów. Odbyna się to 'od końca', to znaczy, że startujemy z każdym punktem jako osobnym klasterem a następnie 'najbardziej podobne' klastry łączymy w jeden dopuki nie otrzymamy zadanej z góry przez użytkownika interesującej nas ich liczby. Na rysunkach 15-18 prezentujemy otrzymane klasteryzacje.

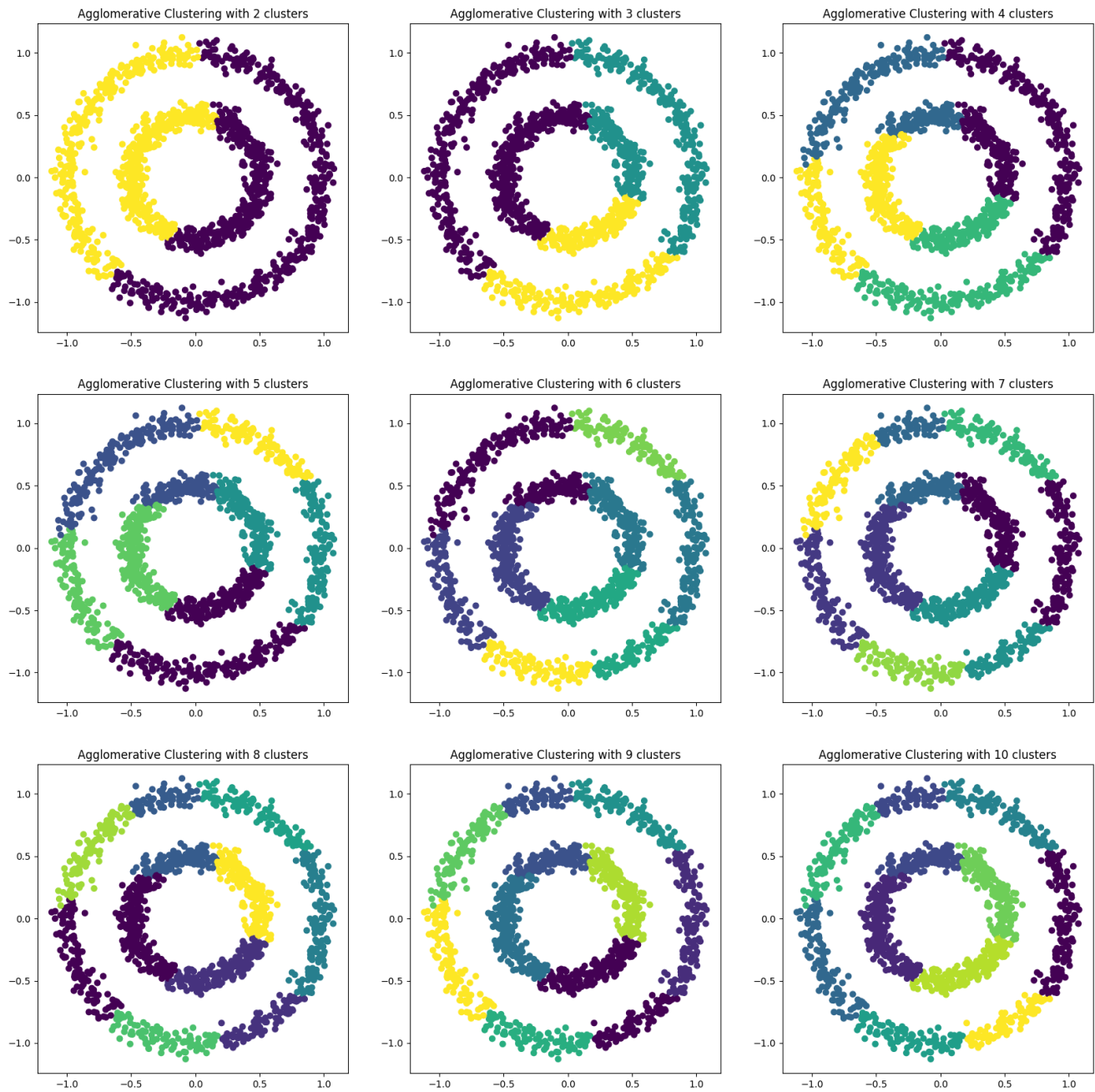
Agglomerative Clustering zawiódł nas swoją skutecznością - pomimo bardziej wyrafinowanego algorytmu dorównuje *KMeans*. Podobnie jak w przypadku *KMeans*, wpływ parametru *noise*, prezentowany na rysunkach 19 i 20, nie jest istotny gdyż nasz algorytm nie radzi sobie z tak skomplikowanymi danymi.

Finalnie prezentujemy ocenę metody *Agglomerative Clustering* przy pomocy metryki *silhouette_score*. Na rysunku 21 widzimy, iż o ile otrzymane wartości są często lepsze niż dla *DBSCAN* to patrząc na reprezentacje graficzne jedynie dla danych *blobs* klasteryzacja przebiegła zgodnie z oczekiwaniami.

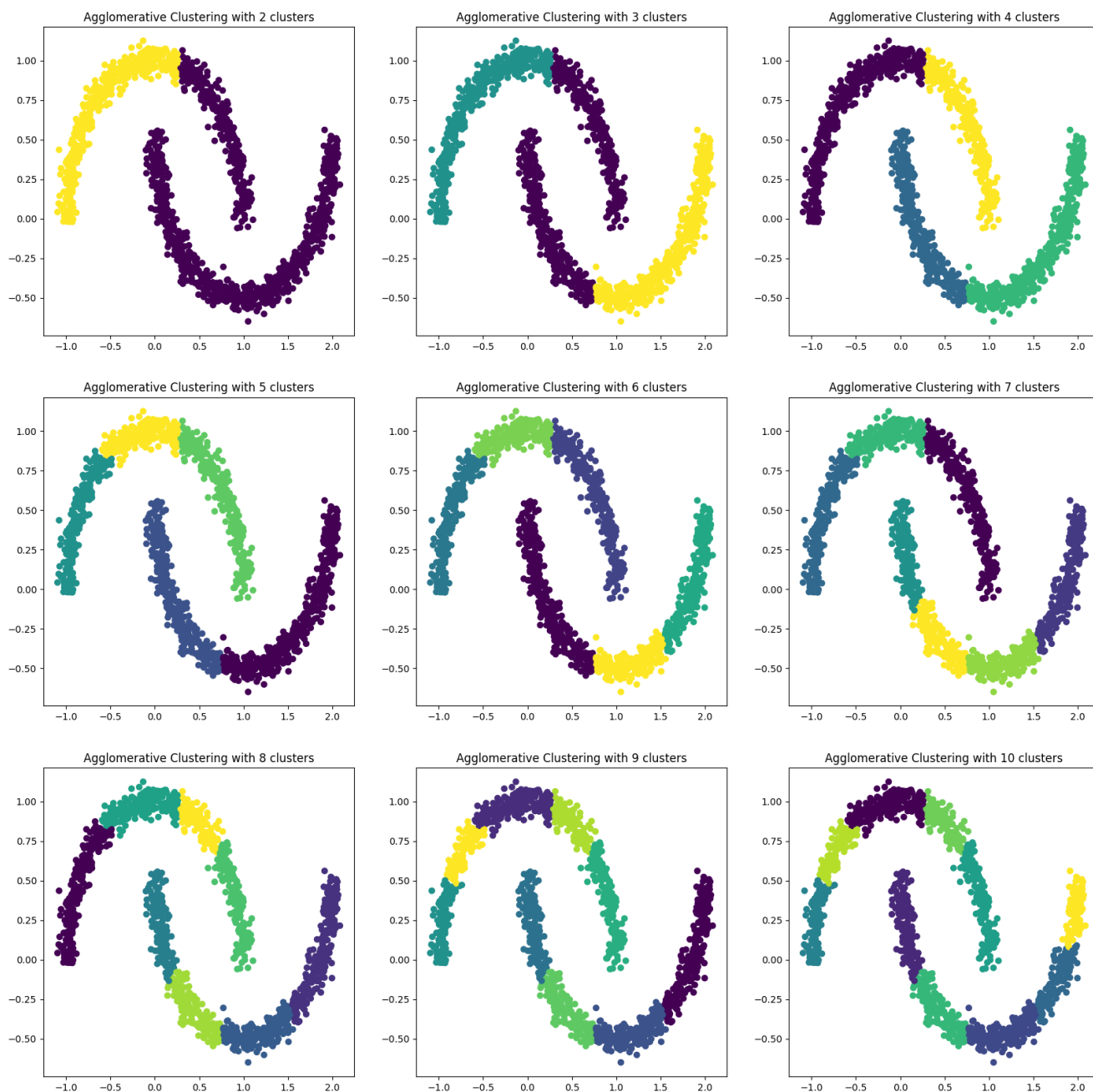
Rysunek 15: Agglomerative Clustering - blobs



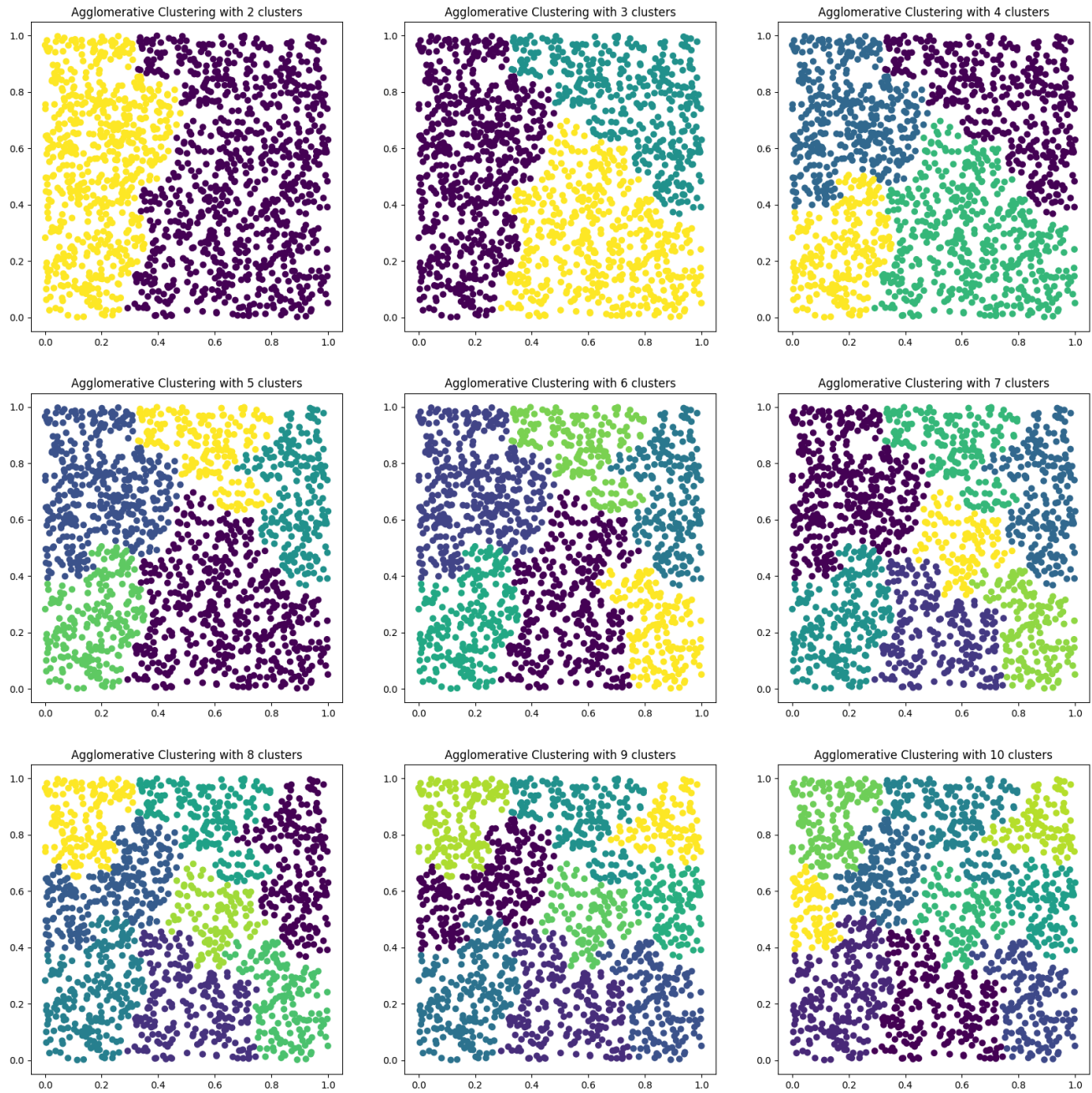
Rysunek 16: Agglomerative Clustering - circles



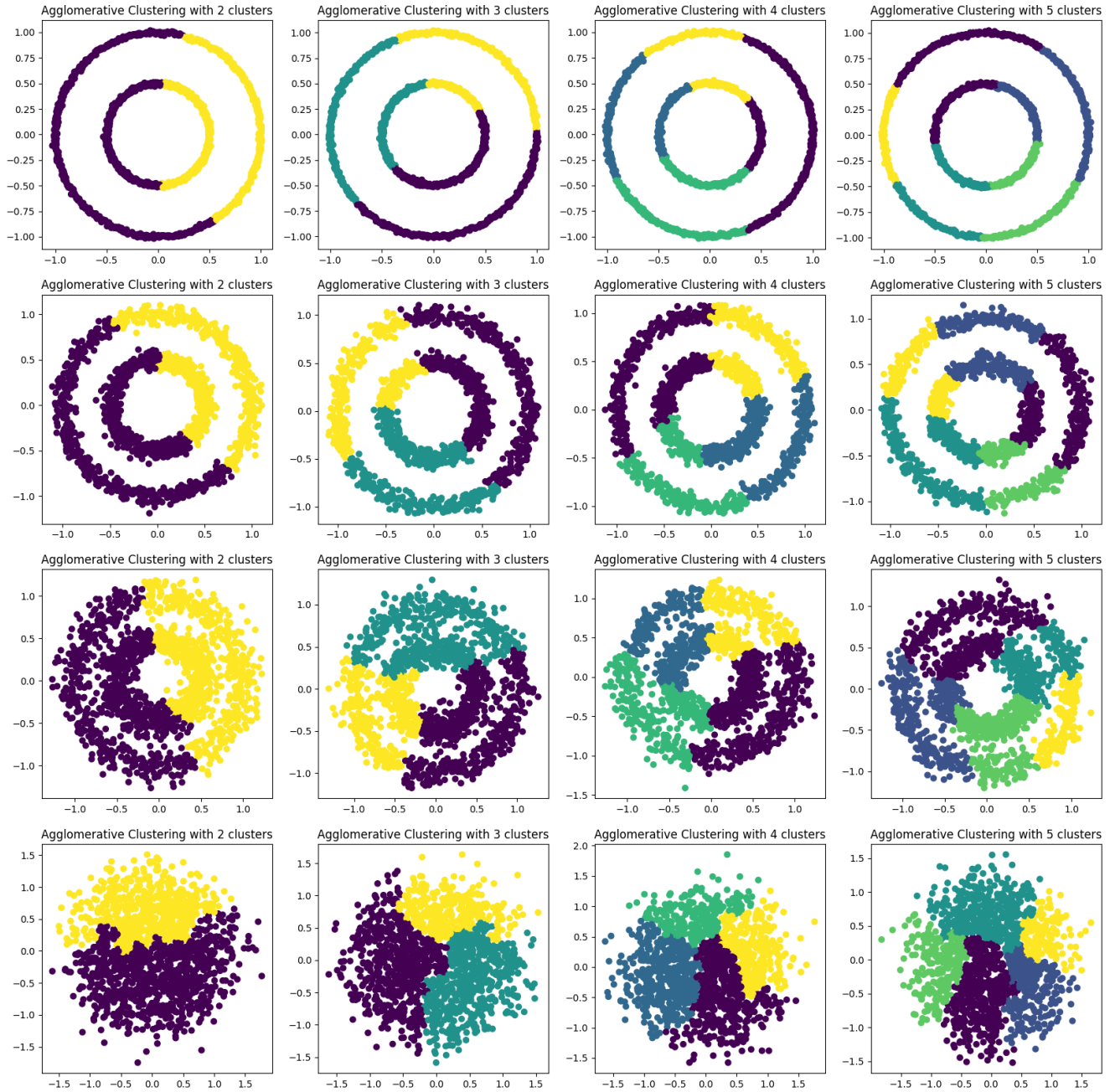
Rysunek 17: Agglomerative Clustering - moons



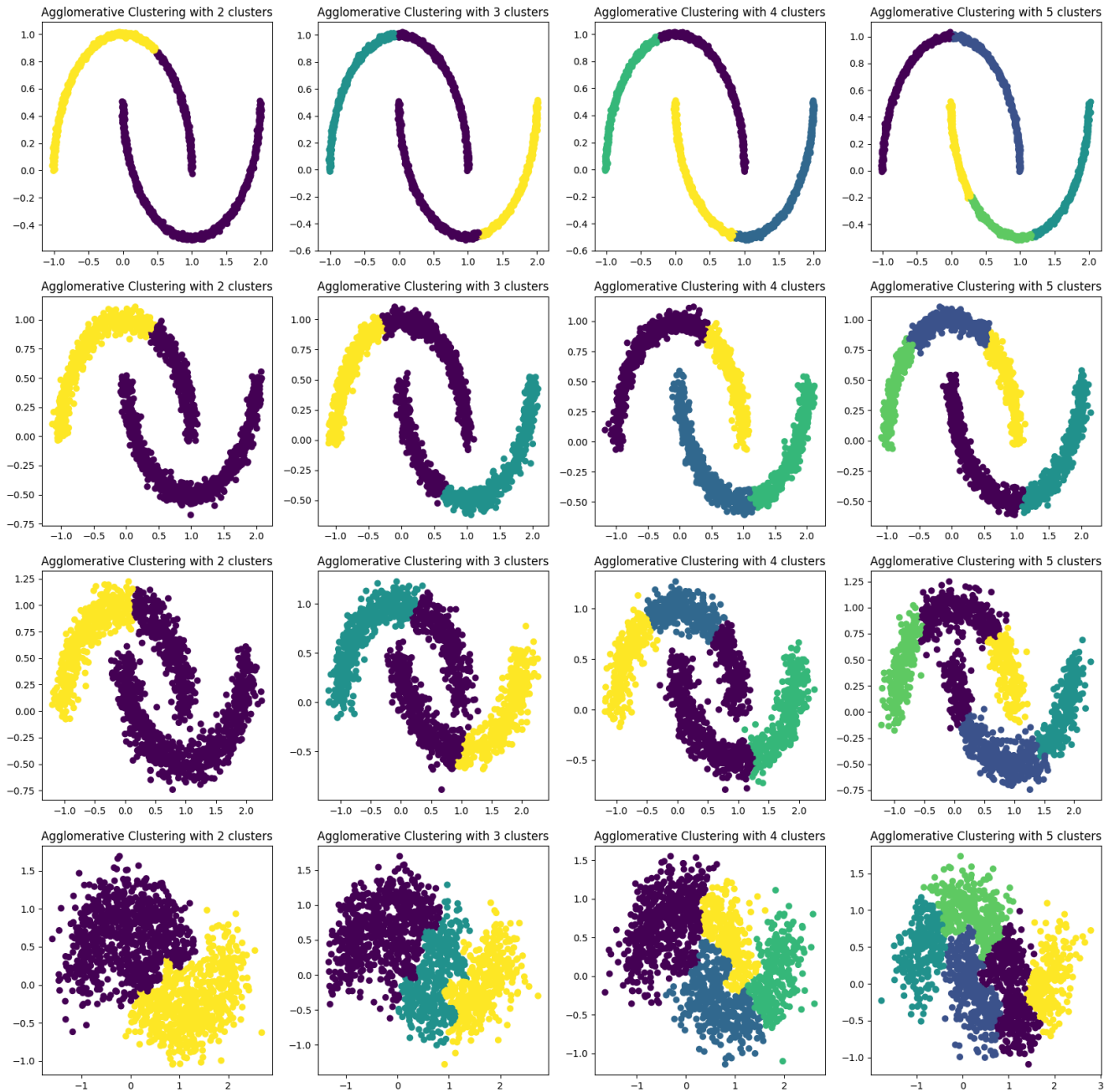
Rysunek 18: Agglomerative Clustering - random



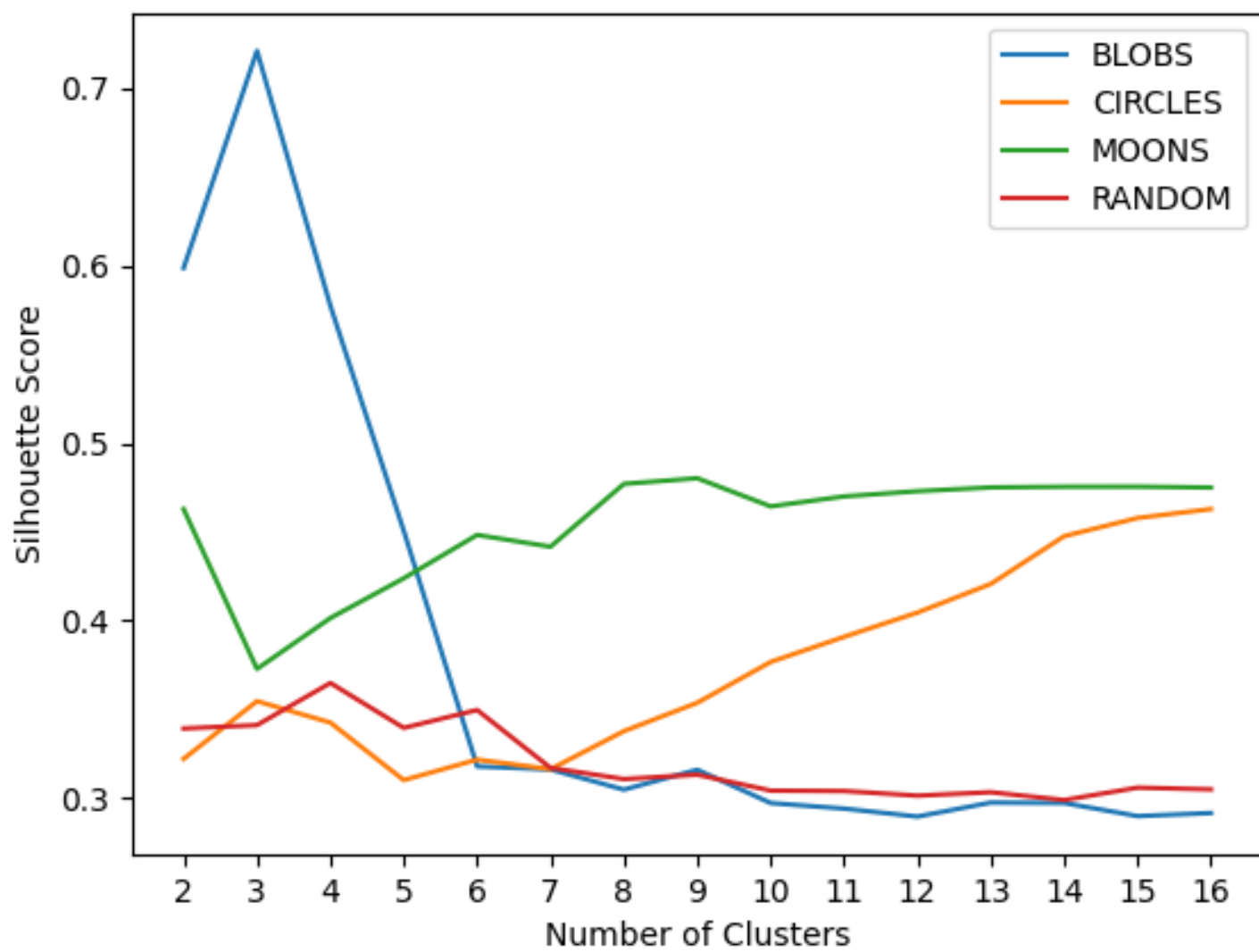
Rysunek 19: Agglomerative Clustering - circles with different noise



Rysunek 20: Agglomerative Clustering - moons with different noise



Rysunek 21: Agglomerative Clustering - score



5 Podsumowanie

'Graficzna zabawa z klastrowaniem' była zdecydowanie przyjemna. Niemniej spodziewamy się, że 'poważne' uczenie nienadzorowane jest zdecydowanie mniej estetyczne wizualnie. Pozytywnie zaskoczył nas algorytm *DBSCAN*, natomiast zawiedliśmy się na *Agglomerative Clustering*. Z niecierpliwością czekamy na to, co przyniosą nam kolejne ML'owe wyzwania, zarówno te akademickie jak i profesjonalne.

Dziękuję za uwagę!